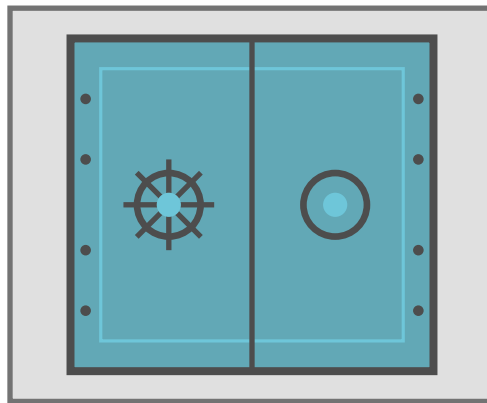


Criptografía y Seguridad

Integridad y autenticación de mensajes

MACs y Cifrado Autenticado



Apunte integral — Teoría + Práctica

Clase 3 · Capítulo 4, *Introduction to Modern Cryptography* (Katz & Lindell)

Instituto Tecnológico de Buenos Aires (ITBA)

Documento elaborado a partir de las transcripciones de la clase teórica (Prof. Pablo Abad) y de la clase práctica, junto con las diapositivas oficiales de ambas.

Índice

1. Motivación: del cifrado a la integridad	2
1.1. Recordatorio: el escenario CPA	2
2. Un nuevo tipo de ataque: maleabilidad	2
2.1. El ataque: no hace falta conocer el valor	3
2.2. Modificaciones quirúrgicas: maleabilidad	3
3. Ataques de texto cifrado escogido (CCA)	4
4. Message Authentication Code (MAC)	5
4.1. Seguridad de un MAC: el experimento Mac-forge	6
5. Ejercicio: ¿son seguros estos MACs?	6
6. Cómo construir un MAC: CBC-MAC	7
6.1. Construcción para mensajes de longitud fija	8
6.2. CBC-MAC (para mensajes más largos, longitud fija)	8
6.3. Ataque a CBC-MAC con longitud variable	8
6.4. Extensiones seguras de CBC-MAC para longitud arbitraria	9
7. Práctica: construcciones para mensajes de longitud variable	9
8. Funciones de hash criptográficas	11
8.1. Colisiones y la propiedad fundamental	11
8.2. Las tres propiedades (niveles de seguridad)	11
8.3. La paradoja del cumpleaños	12
8.4. El modelo iterativo: Merkle-Damgård	12
8.5. Primitivas concretas	13
9. MACs a partir de funciones de hash: HMAC y NMAC	13
9.1. NMAC (Nested MAC)	14
9.2. HMAC (hash-based MAC)	14
10.Privacidad e integridad: combinarlas	14
11.Cifrado Autenticado	15
11.1. Encadenamiento CCM	16
11.2. Encadenamiento GCM	16
12.Aplicaciones prácticas	17
Cross-check teoría vs. práctica	17
Lectura recomendada	17

1 Motivación: del cifrado a la integridad

Hasta esta clase la criptografía vista resolvía un único gran problema: **la confidencialidad** (cómo controlar la diseminación de la información). Esta clase abre un **nuevo campo**: el de la **integridad**, es decir, *cómo confiar en que la información no fue modificada*, aun cuando provenga de una fuente confiable.

Repaso — Criptosistema. Es una terna de algoritmos $\langle \text{Gen}, \text{Enc}, \text{Dec} \rangle$:

$$\text{Gen} : () \rightarrow \mathcal{K}, \quad \text{Enc} : \mathcal{K} \times \mathcal{P} \rightarrow \mathcal{C}, \quad \text{Dec} : \mathcal{K} \times \mathcal{C} \rightarrow \mathcal{P}$$

con la **propiedad básica** $d_k(e_k(m)) = m$ para todo m y k válidos. Conjuntos: $\mathcal{K}$ (claves), $\mathcal{P}$ (planos), $\mathcal{C}$ (cifrados).

1.1 Recordatorio: el escenario CPA

El mejor escenario de seguridad visto hasta ahora era el de **ataque de texto plano escogido** (*Chosen Plaintext Attack*). En su experimento de indistinguibilidad $\text{CPA}_{A,\Pi}$, el adversario A tiene acceso a un **oráculo de cifrado** $f(x) = e_k(x)$ (nunca conoce la clave):

Experimento $\text{CPA}_{A,\Pi}$

1. Se genera una clave $k \leftarrow \mathcal{K}$.
2. A obtiene $f(x) = e_k(x)$ y genera (m_0, m_1) .
3. Se genera $b \leftarrow \{0, 1\}$.
4. A recibe $c = e_k(m_b)$.
5. A emite $b' \in \{0, 1\}$.

$\text{CPA}_{A,\Pi} = 1$ si $b = b'$ (A gana). Si $\Pr[\text{CPA}_{A,\Pi} = 1] = \frac{1}{2} + \varepsilon$ con ε despreciable, Π es **CPA-secure** (indistinguible).

Los criptosistemas **CPA-secure** son no deterministas: incorporan un parámetro extra público y aleatorio, el **IV** (*tweak*), que evita reutilizar la misma semilla del generador. Son CPA-secure los cifrados de flujo con IV y los de bloque con encadenamiento seguro (CBC, Counter, OFB, CFB) sobre primitivas seguras.

2 Un nuevo tipo de ataque: maleabilidad

Escenario. Un sistema de RR.HH. guarda los sueldos de los empleados *cifrados* en una base de datos, con un criptosistema de flujo CPA-secure, para que ni el administrador de la base pueda leerlos.

empleado	sueldo (cifrado)
2542	0xEA26969AA3F61CDD9EC68498DF031CA935EFA9C
2678	0xF4933E107178B88D8EE00F40E43A9A3C9D2EDF79
2789	0x155BBBE7A5442CBBCAB8F57DACF7212255F3641C
2890	0x4D9B47D518F2ED7DE04D601F1856767969A328E8

La parte de cifrado es realmente segura. El atacante no puede extraer información: ni aproximar un sueldo, ni saber si es alto o bajo, ni sacar el número. Aunque le revelen que *uno* de los sueldos vale un millón, como el criptosistema es **no determinista**, puede cifrar el valor “un millón” un millón de veces y nunca obtendrá ninguno de los cuatro textos cifrados de la tabla.

2.1 El ataque: no hace falta conocer el valor

A veces, *para montar un ataque no se necesita conocer el valor*. Supóngase que el atacante conoce su propio legajo (2678) y el de su jefe (2890), y que normalmente el jefe cobra más. Entonces puede tomar el texto cifrado del jefe y hacer un UPDATE copiándolo sobre su propia fila:

empleado	sueldo (cifrado)
2678	0x4D9B47D518F2ED7DE04D601F1856767969A328E8 ¡Aumento!
2890	0x4D9B47D518F2ED7DE04D601F1856767969A328E8

Aumento de sueldo instantáneo, sin haber roto la confidencialidad. Una solución ingenieril es cifrar el par empleado || sueldo y verificar, al descifrar, que el legajo coincide con la fila. *Pero esto no alcanza.*

2.2 Modificaciones quirúrgicas: maleabilidad

Maleabilidad

Un criptosistema es **maleable** cuando una modificación del *texto cifrado* se propaga de forma **sistemática y controlable** al *texto plano*. En su forma más simple: cambio un bit del cifrado y eso cambia un bit del plano.

Los cifrados de flujo son maleables. Si $\text{Enc}_k(m) = G(k) \oplus m = c$, y definimos $c' = c \oplus x$, entonces:

$$\text{Dec}_k(c') = G(k) \oplus c' = G(k) \oplus (c \oplus x) = (G(k) \oplus c) \oplus x = m \oplus x.$$

La modificación x se traslada *directamente* al plano. El atacante conoce el formato:

Formato del campo: IV || empl || sueldo, con IV = 12 bytes, empl = 4 bytes (int), sueldo = 4 bytes (int).

Resolución del ejercicio del backstage. El atacante conoce su propio sueldo en plano (le llega el recibo): 12345. Quiere cambiarlo a 100000 operando *solo* sobre el cifrado. Trabaja sobre los últimos 3 bytes (2EDF79). El procedimiento iterativo (que puede colapsarse en un único XOR triple) es:

	representación binaria			efecto al descifrar
2EDF79	00101110	11011111	01111001	$\text{Dec}_k(c) = 12345$
$\oplus 12345$	00000000	00110000	00111001	$\Rightarrow \text{Dec}_k(c') = 0 \dots 00$
$\oplus 100000$	00000001	10000110	10100000	$\Rightarrow \text{Dec}_k(c'') = 100000$
= 2F69F0	00101111	01101001	11110000	(nuevo cifrado)

- Primer paso ($\oplus 12345$): *anula* el plano dejándolo en ceros ($12345 \oplus 12345 = 0$). Es solo una “plantilla” de partida.

- Segundo paso ($\oplus 100000$): a partir de los ceros, fija el valor deseado ($0 \oplus 100000 = 100000$).

El cifrado resultante **2F69F0** es una jeringoza que nadie entiende (no se tiene la clave), pero al descifrarse da **exactamente** 100 000, y *pasa toda la verificación* del legajo. La maleabilidad es un problema de **integridad**, no resuelto por el cifrado.

3 Ataques de texto cifrado escogido (CCA)

El formalismo que captura “protegernos de cualquier modificación del cifrado” es el **ataque de texto cifrado escogido** (*Chosen Ciphertext Attack*). Es una versión más paranoica de CPA: el adversario tiene ahora **también un oráculo de descifrado**.

Experimento $\text{CCA}_{A,\Pi}$

Dado un nivel de seguridad n , un adversario A y un criptosistema $\Pi(n)$:

1. Se genera una clave $k \leftarrow \mathcal{K}$.
2. A obtiene $f(x) = e_k(x)$ y $g(x) = d_k(x)$, y genera (m_0, m_1) .
3. Se genera $b \leftarrow \{0, 1\}$.
4. A recibe $c = e_k(m_b)$, y **no puede calcular** $g(c)$ (sino el ataque sería trivial).
5. A emite $b' \in \{0, 1\}$.

$\text{CCA}_{A,\Pi} = 1$ si $b = b'$. Si $\Pr[\text{CCA}_{A,\Pi} = 1] < \frac{1}{2} + \text{negl}(n)$, Π es **CCA-secure**.

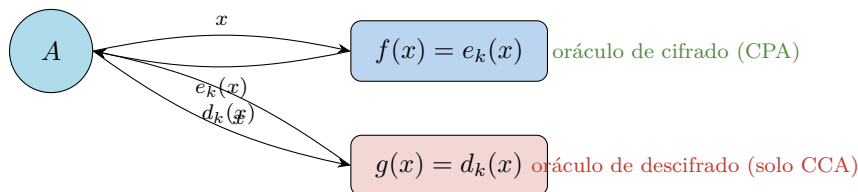


Figura 1: En CCA el adversario suma el *oráculo de descifrado* g al de cifrado f del escenario CPA. (Recreación conceptual de las diapositivas.)

Ningún criptosistema visto es CCA-secure

La buena noticia: CCA es una prueba **abarcativa** que cubre todas las familias de modificaciones del cifrado (no hay que jugar al gato y al ratón). La mala: **ninguno** de los criptosistemas vistos (ni los no vistos) la pasa, porque el cifrado está pensado para *confidencialidad* (transferencia de información), no para *integridad* (manipulación de la información).

Ejercicio (slide): el cifrado de flujo no es CCA-secure

Enunciado. Demostrar que el cifrado de flujo en general no es CCA-secure. *Ayuda:* considerar $m_0 = (0 \dots 0)$ y $m_1 = (1 \dots 1)$ y verificar qué consultas se pueden hacer a $g(x)$ al recibir c .

Resolución. El adversario propone $m_0 = 0^n$ y $m_1 = 1^n$ y recibe el challenge $c = e_k(m_b) = G(k) \oplus m_b$. No puede preguntar $g(c)$, pero sí $g(c')$ para cualquier $c' \neq c$. Construye $c' = c \oplus 1^n$ (flip de todos los bits). Por maleabilidad:

$$g(c') = \text{Dec}_k(c \oplus 1^n) = m_b \oplus 1^n = \overline{m_b}.$$

Si $b = 0$ entonces $m_b = 0^n$ y $g(c') = 1^n$; si $b = 1$ entonces $g(c') = 0^n$. El adversario lee $g(c')$ y deduce b con **probabilidad 1**. Como $c' \neq c$, la consulta es legal. ■

Esto requiere **control de integridad**: identificar adulteraciones. La primitiva criptográfica para ello es el **MAC** (*Message Authentication Code*).

4 Message Authentication Code (MAC)

Definición formal de MAC

Un MAC es una **terna de algoritmos** $\langle \text{Gen}, \text{Mac}, \text{Vrfy} \rangle$:

$\text{Gen} : () \rightarrow \mathcal{K}$ (generador de clave), $\text{Mac} : \mathcal{K} \times \mathcal{P} \rightarrow \mathcal{T}$ (etiquetador),

$\text{Vrfy} : \mathcal{K} \times \mathcal{P} \times \mathcal{T} \rightarrow \{0, 1\}$ (verificador).

Propiedad básica: para todo m y k válidos, $\text{Vrfy}_k(m, \text{Mac}_k(m)) = 1$.

Conjuntos: $\mathcal{K}$ (claves), $\mathcal{P}$ (planos), $\mathcal{T}$ (etiquetas/tags).

El emisor envía el par $\langle m, t \rangle$ con $t = \text{Mac}_k(m)$; el receptor calcula $b = \text{Vrfy}_k(m, t)$ y acepta si $b = 1$. La función **Mac** parte de un mensaje plano arbitrario y una clave y genera una etiqueta; **Vrfy** recibe clave, mensaje y etiqueta y devuelve un booleano.

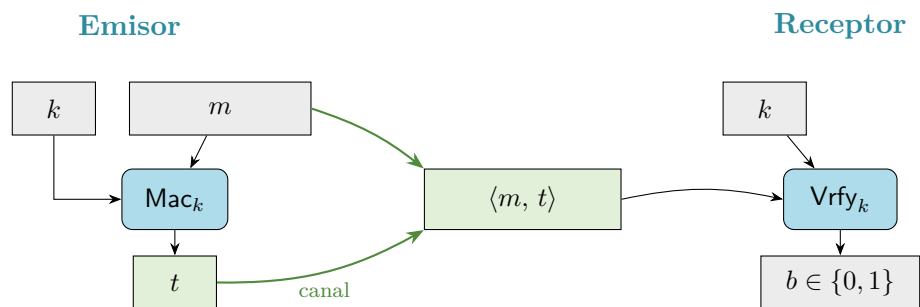


Figura 2: Generación y verificación de un MAC. El emisor adjunta $t = \text{Mac}_k(m)$; el receptor acepta si $\text{Vrfy}_k(m, t) = 1$. (*Recreación de las diapositivas teórica y práctica.*)

Aclaraciones del profesor

- **Un MAC no provee no-repudio.** La clave k es **simétrica** (compartida por emisor y receptor): cualquiera de los dos pudo generar la etiqueta, así que ante un tercero no se puede probar *quién* la emitió. El no-repudio requiere firma digital (clave pública), que se ve más adelante.
- **Un MAC no protege contra ataques de replay.** Una etiqueta válida sigue siendo válida si se reenvía. Se mitiga incorporando un **número de secuencia** o un **timestamp** dentro del mensaje autenticado.
- **MAC $\neq$ dirección MAC** de redes: aquí es *Message Authentication Code*.

4.1 Seguridad de un MAC: el experimento Mac-forge

Experimento $\text{Mac-forge}_{A,\Pi}$

Dado un nivel de seguridad n , un adversario A y un esquema $\Pi(n)$:

1. Se genera una clave $k \leftarrow \text{Gen}(1^n)$.
2. A recibe 1^n y acceso al **oráculo** $f(x) = \text{Mac}_k(x)$. Realiza todas las evaluaciones que quiera (sea Q el conjunto de consultas).
3. A emite un par $\langle m, t \rangle$ con $m \notin Q$.

$\text{Mac-forge}_{A,\Pi} = 1$ (**éxito**) si y solo si $\text{Vrfy}_k(m, t) = 1$ y $m \notin Q$.

El MAC es **seguro (infalsificable)** ante un ataque de mensaje elegido si y solo si, para todo adversario PPT A :

$$\Pr[\text{Mac-forge}_{A,\Pi}(n) = 1] \leq \text{negl}(n).$$

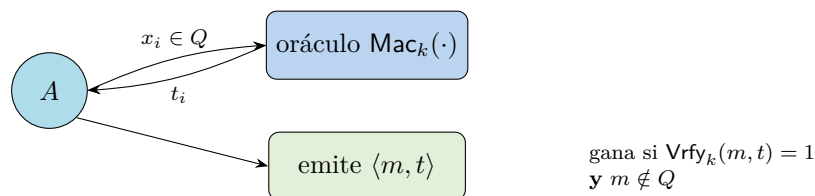


Figura 3: El adversario consulta libremente el oráculo (conjunto Q) y debe forjar una etiqueta válida para un mensaje **nuevo** $m \notin Q$.

Observaciones. (i) El adversario puede obtener un MAC para *cualquier* mensaje que elija. (ii) Se considera **roto** el MAC si el adversario puede falsificar cualquier mensaje, **independientemente de si tiene sentido o no**.

5 Ejercicio: ¿son seguros estos MACs?

La siguiente serie de MACs “ingenuos” permite extraer las **propiedades que un MAC seguro debe cumplir**. Todos se rompen con Mac-forge. Resolvemos cada uno con el paso a paso de la clase.

(a) $\text{Mac}_k(m) = G(k) \oplus m$ — usar un cifrado de flujo como MAC

G es un generador pseudoaleatorio: la etiqueta es tan larga como el mensaje y resulta de hacer XOR contra la secuencia $G(k)$.

Problema: la salida $G(k)$ es *determinística* (con la misma clave, el primer bloque del generador es siempre el mismo). Etiquetar es simplemente hacer XOR contra $G(k)$, así que si el adversario **averigua $G(k)$ puede etiquetar cualquier mensaje**.

Algoritmo atacante (Mac-forge):

1. Consulta el oráculo con $m = 0 \dots 0$. Obtiene $t_0 = \text{Mac}_k(0) = G(k) \oplus 0 = \boxed{G(k)}$. ¡Recuperó la corriente de clave!
2. Elige un mensaje nuevo $m' \neq 0$, por ejemplo $m' = 1 \dots 1$, y calcula $t' = G(k) \oplus m'$.
3. Emite $\langle m', t' \rangle$ con $m' \notin Q$: pasa la verificación. ■

(b) $\text{Mac}_k(m) = k \oplus \text{first_k_bits}(m)$

La etiqueta es el XOR de la clave (longitud fija) con los *primeros* $|k|$ bits del mensaje. *Hay dos falsificaciones distintas:*

Ataque 1 — recuperar la clave (la del curso).

1. Consulta $m = 0 \dots 0$: $t = k \oplus \text{first_k_bits}(0) = k \oplus 0 = \boxed{k}$. ¡Recuperó la clave entera!
2. Emite un mensaje nuevo, p. ej. $m' = 1 \dots 1$ con etiqueta $t' = k \oplus \underbrace{(1 \dots 1)}_{k \text{ bits}}$. Pasa la verificación. ■

Ataque 2 — más fácil aún (la lección buscada). El MAC **ignora** todo el mensaje salvo sus primeros $|k|$ bits. Entonces:

1. Consulta un mensaje m y obtiene su etiqueta t .
2. Emite $\langle m \parallel \text{<carácter extra>, } t \rangle$: el sufijo agregado cae fuera de los primeros $|k|$ bits, así que la **misma etiqueta** sigue siendo válida para un mensaje distinto. ■

Lección: *un MAC que no considera la totalidad del mensaje es trivialmente falsificable.* Si ignora algún bit, basta modificar ese bit y reusar la etiqueta.

(c) $\text{Mac}_k(m) = \text{Enc}_k(|m|)$ con Enc CPA-secure

La etiqueta cifra **solo la longitud** del mensaje, no su contenido.

Verificación: hay que descifrar la etiqueta y ver si la longitud coincide. **Problema:** la etiqueta *no depende del contenido*.

Algoritmo atacante:

1. Pide el etiquetado de un mensaje m_A de longitud L : obtiene t_A .
2. Toma cualquier *otro* mensaje $m_B \neq m_A$ de la **misma longitud** L .
3. Emite $\langle m_B, t_A \rangle$. Al verificar, se descifra $t_A \Rightarrow L$, que coincide con $|m_B|$: **válido**. ■

(Como Enc es no determinista, $t_A \neq t_B$ aunque $|m_A| = |m_B|$; pero ambos descifran a la misma longitud, así que las etiquetas son intercambiables.)

Conclusión. Un MAC robusto/infalsificable es *siempre* el resultado de procesar la **totalidad del contenido** del mensaje, sin determinismos triviales extraíbles ni partes ignoradas.

6 Cómo construir un MAC: CBC-MAC

Primera familia de construcciones: **a partir de una primitiva de cifrado de bloque**. Sea F una función pseudoaleatoria ($F_k : \{0, 1\}^n \rightarrow \{0, 1\}^n$).

6.1 Construcción para mensajes de longitud fija

MAC seguro para mensajes de longitud fija n

Gen: $k \leftarrow \{0,1\}^n$. **Mac:** $t \leftarrow F_k(m)$ con $|m| = |k| = n$. **Vrfy:** si $|m| \neq |k| \Rightarrow 0$; si $F_k(m) = t \Rightarrow 1$.

Como F_k es una PRF de n bits a n bits que procesa *todo* el bloque, esta construcción es infalsificable **para mensajes de longitud fija**.

6.2 CBC-MAC (para mensajes más largos, longitud fija)

CBC-MAC

Gen: $k \leftarrow \{0,1\}^n$. **Mac:** dado $m = m_1 \parallel m_2 \parallel \dots \parallel m_\ell$ (bloques de n bits), con $t_0 = 0^n$:

$$t_i = F_k(t_{i-1} \oplus m_i), \quad \text{Mac}_k(m) = t_\ell.$$

Vrfy: $\text{Vrfy}_k(m, t) = 1 \iff t = \text{Mac}_k(m)$.

A diferencia del CBC *de cifrado*, aquí se emite **una sola etiqueta** (la del último bloque) y **no hay IV** (el estado inicial es fijo en 0^n). Se encadena un bloque con el siguiente; importa el orden y los estados intermedios $t_1, \dots, t_{\ell-1}$ **se descartan**.

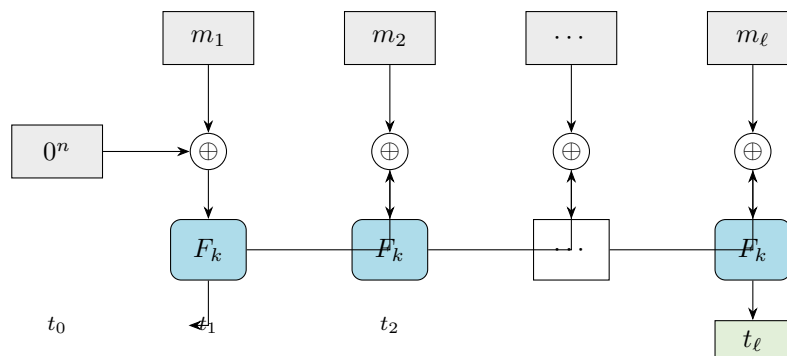


Figura 4: CBC-MAC: encadenamiento $t_i = F_k(t_{i-1} \oplus m_i)$ con $t_0 = 0^n$; la etiqueta es el **último** t_ℓ . (Recreación de las diapositivas.)

Aclaración clave del profesor

La construcción CBC-MAC anterior es **infalsificable SOLO si se permiten mensajes de una misma longitud**. Con mensajes de **longitud variable** es falsificable.

6.3 Ataque a CBC-MAC con longitud variable

El atacante explota que los **estados intermedios** ($t_1, \dots$) se descartan, pero puede recalcularlos: el estado tras procesar dos bloques, $F_k(\cdot) \oplus m_3$, le permite “empalmar” mensajes.

Ataque (slide teórica): forja con longitud variable

1. Crear dos bloques aleatorios A y B . Definir $m_1 = A \parallel B$ y $m_2 = A$.
2. Consultar el oráculo: $t_1 = f(m_1)$, $t_2 = f(m_2)$.

3. **Observar:** $t_2 = F_k(0^n \oplus A) = F_k(A)$, y $t_1 = F_k(F_k(A) \oplus B) = F_k(t_2 \oplus B)$.
4. Emitir el mensaje de **tres** bloques $m^* = A \parallel B \parallel (A \oplus t_1)$ con etiqueta t_2 .

Por qué funciona. Calculamos $\text{Mac}_k(m^*)$ paso a paso:

$$\begin{aligned} s_1 &= F_k(0^n \oplus A) = F_k(A) = t_2, \\ s_2 &= F_k(s_1 \oplus B) = F_k(t_2 \oplus B) = t_1, \\ s_3 &= F_k(s_2 \oplus (A \oplus t_1)) = F_k(t_1 \oplus A \oplus t_1) = F_k(A) = t_2. \end{aligned}$$

Luego $\text{Mac}_k(m^*) = s_3 = t_2$, que es justamente la etiqueta emitida, y $m^* \notin Q$. **Falsificación válida.** ■

6.4 Extensiones seguras de CBC-MAC para longitud arbitraria

Para autenticar mensajes de **cualquier** longitud existen tres opciones seguras (y una insegura para descartar):

Opciones seguras

Opción 1 — derivar clave de la longitud

$k' = F_k(|m|)$ y $t \leftarrow \text{CBC-MAC}_{k'}(m)$. (Cada longitud usa una clave distinta.)

Opción 2 — longitud como prefijo

$m' = |m| \parallel m$ y $t \leftarrow \text{CBC-MAC}_k(m')$. La longitud debe ir al principio, no al final.

Opción 3 — doble clave

$k_1, k_2 \leftarrow \{0, 1\}^n$; $t \leftarrow \text{CBC-MAC}_{k_1}(m)$; $\hat{t} \leftarrow F_{k_2}(t)$.

La longitud como *sufijo* NO es segura

Si $m' = m \parallel |m|$ y $\text{Mac}_k(m) = \text{CBC-MAC}_k(m')$, hay un ataque. Con bloques aleatorios A, B, C :

$$\begin{aligned} m_1 &= AAA \rightarrow m'_1 = AAA3, & t_1 &= f(m_1) \\ m_2 &= BBB \rightarrow m'_2 = BBB3, & t_2 &= f(m_2) \\ m_3 &= AAA3CC \rightarrow m'_3 = AAA3CC6, & t_3 &= f(m_3) \end{aligned}$$

Sea $X = t_1 \oplus t_2 \oplus C$; emitir $(BBB3XC, t_3)$.

La etiqueta t_3 resulta válida para un mensaje no consultado. Por eso la longitud va como *prefijo*. (Notación de la slide; 3, 6 representan los sufijos de longitud.)

7 Práctica: construcciones para mensajes de longitud variable

La clase práctica desarrolla **en detalle** por qué las construcciones “naturales” para extender un MAC de bloque fijo Mac' a mensajes de longitud variable fallan, antes de llegar a la solución (ineficiente pero segura) y, finalmente, a CBC-MAC.

Sugerencia 1 — XOR de los bloques y autenticar el resultado

$$t := \text{Mac}'_k\left(\bigoplus_i m_i\right), \quad \text{emitir } \langle m_1, m_2, \dots, m_d; t \rangle.$$

Falla. El adversario puede falsificar un tag válido sobre un *nuevo* mensaje cambiando el mensaje original **de modo que el XOR de los bloques no cambie**. Por ejemplo, mueve un bit de un bloque a otro: si el XOR total da lo mismo, la etiqueta sigue siendo válida y $m' \neq m$. ■

Sugerencia 2 — autenticar cada bloque por separado

$$t_i := \text{Mac}'_k(m_i), \quad \text{emitir } \langle m_1, m_2, \dots, m_d; t_1, t_2, \dots, t_d \rangle.$$

Falla. El adversario consulta cada bloque por separado (Mac' de m_1 , de m_2 ...) y luego **reordena/permuta** los bloques con sus etiquetas. Por ejemplo, de $\langle m_1, m_2, \dots; t_1, t_2, \dots \rangle$ arma $\langle m_2, m_1, m_3; t_2, t_1, t_3 \rangle$: cada par (m_i, t_i) valida bien, así que el mensaje permutado pasa. También puede **mezclar bloques de mensajes distintos**. ■

Sugerencia 3 — autenticar cada bloque con su número de secuencia

$$t_i := \text{Mac}'_k(i \parallel m_i), \quad \text{emitir } \langle m_1, \dots, m_d; t_1, \dots, t_d \rangle.$$

Mejora pero aún falla. El número de secuencia impide reordenar (ahora cada t_i depende de la posición i). **Pero** el adversario todavía puede **mezclar bloques de mensajes diferentes que estén en la misma posición**: si consulta dos mensajes y toma m_1 del primero, m'_2 del segundo, m_3 del primero... cada $(i \parallel m)$ sigue correspondiendo a su tag válido $\Rightarrow$ el mensaje híbrido valida. ■

Construcción segura (ineficiente): random + longitud + secuencia + bloque

A partir de $\text{Mac}' = \langle \text{Gen}', \text{Mac}', \text{Vrfy}' \rangle$ para mensajes de longitud n :

- **Gen:** Gen' , con $|k| = n$.
- **Mac:** para $|m| = L < 2^{n/4}$, se parte m en d bloques de $n/4$ bits (se completa con ceros) y se elige $r \leftarrow \{0, 1\}^{n/4}$:

$$t_i = \text{Mac}'_k(r \parallel L \parallel i \parallel m_i), \quad i = 1, \dots, d, \quad \text{emitir } \langle r, t_1, \dots, t_d \rangle.$$

Por qué es seguro:

- **El número r aleatorio** es el *mismo* para todos los bloques de un mismo mensaje y *distinto* entre mensajes. No se pueden **mezclar bloques de mensajes distintos**: tendrían valores de r que no coinciden y la verificación falla (al armar un mensaje híbrido, ¿qué r emitir en el tag? No hay uno que sirva para todos).
- **La longitud L** hace saber cuántos bloques deben llegar (impide truncamientos/extensiones).
- **El número de secuencia i** mantiene el orden (impide permutar).

Problema: ¡ineficiente! Emite *todos* los tags por separado (una barbaridad de pedacitos) y además parte el mensaje en bloques de $n/4$. Es seguro pero poco práctico $\Rightarrow$ se usa **CBC-MAC**, que emite una sola etiqueta.

8 Funciones de hash criptográficas

Función de hash

Son **pares de algoritmos** $\langle \text{Gen}, H \rangle$:

$$\text{Gen} : s \leftarrow \mathcal{S}, \quad h = H^s(m) \in \{0,1\}^L,$$

con L la longitud (**fija**) del hash. **Comprimen** una entrada de longitud arbitraria a una salida de longitud fija (por eso, *funciones de resumen / digest*).

Diferencia importante con un MAC. s **no es una clave**: es un mero *selector* (público) de la función dentro de la familia. En muchas implementaciones $\mathcal{S} = \{s_0\}$ (hay una única función). Las funciones de hash son **análogas a los MACs, pero sin clave**.

8.1 Colisiones y la propiedad fundamental

Colisión

Una **colisión** es un par $x \neq x'$ con $H(x) = H(x')$.

Principio del palomar: si hay $n+1$ mensajes y n valores de salida posibles, existe al menos una colisión. *Las colisiones van a existir siempre* (la entrada es más grande que la salida); el objetivo es que sean **computacionalmente difíciles** de encontrar.

8.2 Las tres propiedades (niveles de seguridad)

Resistencia a preimágenes	Dado y , es computacionalmente imposible hallar x tal que $h(x) = y$. (<i>One-way</i> .)
Resistencia a segundas preimágenes	Dado x , es imposible hallar $x' \neq x$ con $h(x') = h(x)$.
Resistencia a colisiones	Es imposible hallar <i>cualquier</i> par x, x' con $h(x) = h(x')$ y $x \neq x'$.

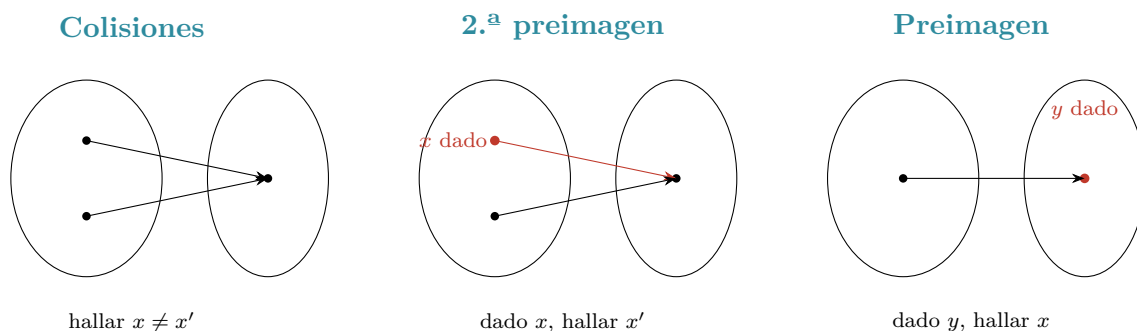


Figura 5: Los tres niveles de seguridad. La **resistencia a colisiones** es donde el atacante tiene *mayor* ventaja (mayor grado de libertad: elige ambos x, x'); implica las otras dos. (*Recreación de las diapositivas.*)

Experimento Hash-Coll_{A,H}

Dado un nivel n , un adversario A y una familia $H^s(n)$:

1. Se selecciona una función $s \leftarrow \mathcal{S}$.
2. A obtiene acceso a $H(x) = H^s(x)$.
3. A emite x, x' .

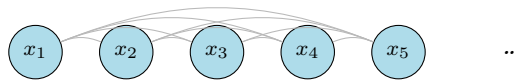
Hash-Coll_{A,H} = 1 si $x \neq x'$ y $H(x) = H(x')$. Si $\Pr[\text{Hash-Coll}_{A,H} = 1] < \text{negl}(n)$, H es **libre de colisiones**. **El desafío:** s se fija en el momento, nada precalculado, y en tiempo polinómico hay que materializar dos mensajes que colisionen.

8.3 La paradoja del cumpleaños

Costo de los ataques por fuerza bruta. Sea $h : A \rightarrow B$.

- **Preimágenes:** requiere $|B|$ intentos (en promedio).
- **Segundas imágenes:** requiere $|B|$ intentos.
- **Colisiones:** requiere solo $|B|^{1/2}$ intentos. **Paradoja del cumpleaños.**

¿Por qué la raíz cuadrada? Buscando una colisión arbitraria, el atacante genera un mensaje, lo compara contra *todos* los anteriores; genera otro y lo compara con todos los previos... Cada nuevo mensaje se chequea contra el conjunto acumulado, de modo que la cantidad de *pares* crece cuadráticamente. La complejidad cae a $\sqrt{|B|}$: medido en bits, es **la mitad del nivel de seguridad** que las otras propiedades. Es la misma optimización que en probabilidad: una cosa es la chance de que alguien coincida con *una* fecha fija, otra es que *cualquiera* coincida con *cualquiera* en un grupo.



Cada nuevo mensaje se compara con *todos* los anteriores:
 $\sim \binom{q}{2}$ pares $\Rightarrow$ colisión esperada con $q \approx \sqrt{|B|}$.

Figura 6: Paradoja del cumpleaños: la cantidad de pares crece como q^2 , reduciendo el costo de hallar una colisión a $\sqrt{|B|}$. (*Recreación conceptual.*)

8.4 El modelo iterativo: Merkle-Damgård

Propuesto por **Merkle (1989)** y usado por muchas funciones. Procesa entradas de longitud arbitraria a partir de una función de **compresión** h^s de bloque pequeño.

- **Preprocesamiento:** ajustar el tamaño del mensaje (*padding*) y agregar un bloque con la longitud.
- **Función de compresión f :** similar al hash pero sobre bloques pequeños; **la seguridad del esquema está dada por la función de compresión** (si h^s es mala, todo el esquema es malo).

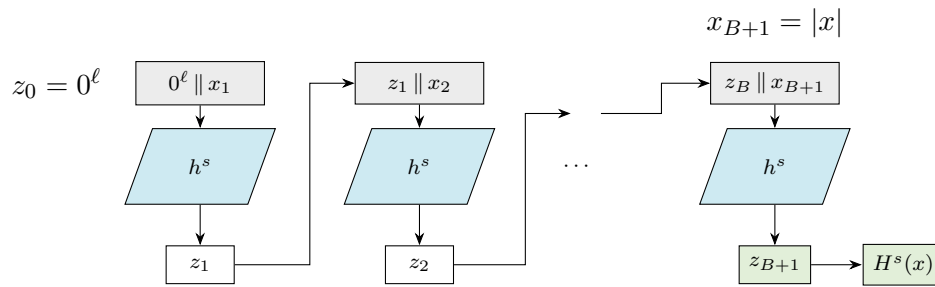


Figura 7: Transformación de Merkle-Damgård: $z_i = h^s(z_{i-1} || x_i)$, con $z_0 = 0^\ell$ y el bloque final $x_{B+1} = |x|$. **Una colisión en H^s solo puede ocurrir si hay una colisión en h^s .** (*Recreación de las diapositivas.*)

8.5 Primitivas concretas

Primitiva	Salida	Notas
MD5	128 bits	Quebrada ; aún aparece en protocolos viejos.
SHA-1	160 bits	Se construye sobre MD5; <i>al borde</i> , complicada.
SHA-2	256/384/512 bits	Estándar durante mucho tiempo.
SHA-3	224/256/384/512 bits	Estandarizado por NIST (2013), antes <i>Keccak</i> ; usa modelo esponja . Recomendado para proyectos nuevos.

Ejemplos (entrada → digest hex).

MD5: "" → d41d8cd98f00b204e9800998ecf8427e; "abc" → 900150983cd24fb0d6963f7d28e17f72

SHA-1: "" → da39a3ee5e6b4b0d3255bfef95601890afd80709; "abc" → a9993e364706816aba3e25717850c26c9cd0d89d

Tamaño mínimo en la práctica

El tamaño mínimo de salida usado hoy es **160 bits**, para que la búsqueda de colisiones requiera más de 2^{80} operaciones (preimágenes/ 2^a imágenes: 2^{160}). **Condición necesaria pero NO suficiente:** aunque la salida sea grande, si la función es mala el esquema es inseguro. Las funciones se miden por el nivel de seguridad de la *resistencia a colisiones* (la mitad de los bits).

9 MACs a partir de funciones de hash: HMAC y NMAC

Segunda familia de construcciones de MAC: **a partir de una función de hash** libre de colisiones. Ventaja sobre CBC-MAC: las funciones de hash están diseñadas para procesar volúmenes grandes de datos muy rápido. Un MAC basado en hash suele ser **1 o 2 órdenes de magnitud más rápido** que un CBC-MAC.

9.1 NMAC (Nested MAC)

NMAC

Gen: emite (s, k_1, k_2) con $|k_1| = |k_2| = n$. **Mac:** para $m = m_1 m_2 \dots m_B$ con $|m| = L$,

$$t := h_{k_1}^s(H_{k_2}^s(m)).$$

Se hace el hash con clave del mensaje (clave k_2 y la longitud L como bloque final) obteniendo $z = H_{k_2}^s(m)$, y luego un *último* paso con la otra clave k_1 : $t = h_{k_1}^s(z)$.

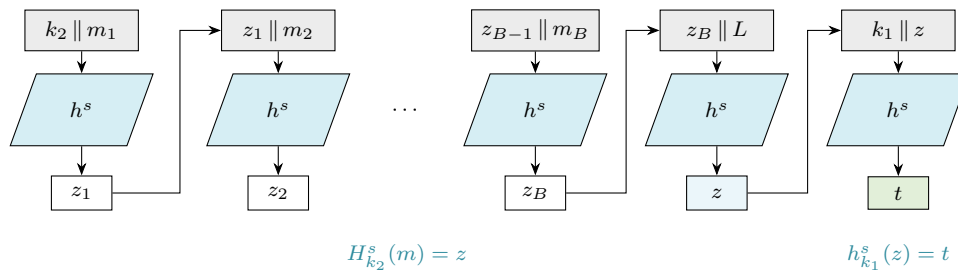


Figura 8: NMAC: hash con clave k_2 (y la longitud L como bloque final) seguido de un paso con k_1 . (Recreación de la diapositiva práctica.)

9.2 HMAC (hash-based MAC)

La construcción de HMAC está especificada en una RFC y es la más usada.

HMAC

Gen: emite (s, k) con $|k| = n$.

$$t := H_{IV}^s\left((k \oplus \text{opad}) \parallel H_{IV}^s((k \oplus \text{ipad}) \parallel m)\right),$$

con $\text{ipad} = 0x36 \dots 36$, $\text{opad} = 0x5c \dots 5c$ (constantes de n bytes) e IV una constante fija de longitud n .

Es infalsificable (Mac-forge) si H es libre de colisiones.

Eficiencia. Aunque a primera vista parezca que se procesa el mensaje dos veces, en realidad m se procesa **una sola vez** en el hash interno; el paso externo (con opad) solo procesa la etiqueta intermedia z (longitud fija). Por eso es muy eficiente. El prefijo $k \oplus \text{ipad}$ **rompe el patrón** entre mensajes iguales y los estados intermedios.

10 Privacidad e integridad: combinarlas

Para lograr **privacidad e integridad** a la vez hay que *combinar* cifrado y etiquetado. Hay **tres formas**, pero **solo dos son seguras**.

- (1) **Cifrar y autenticar** (Mac sobre el *plano*): **inseguro**. La etiqueta t puede brindar información sobre m (un MAC no garantiza confidencialidad de su entrada; dos mensajes iguales podrían dar la misma etiqueta).
- (2) **Autenticar luego cifrar** (Mac sobre el plano, todo cifrado): **puede ser seguro**, pero requiere prueba (problemas tipo *padding*).

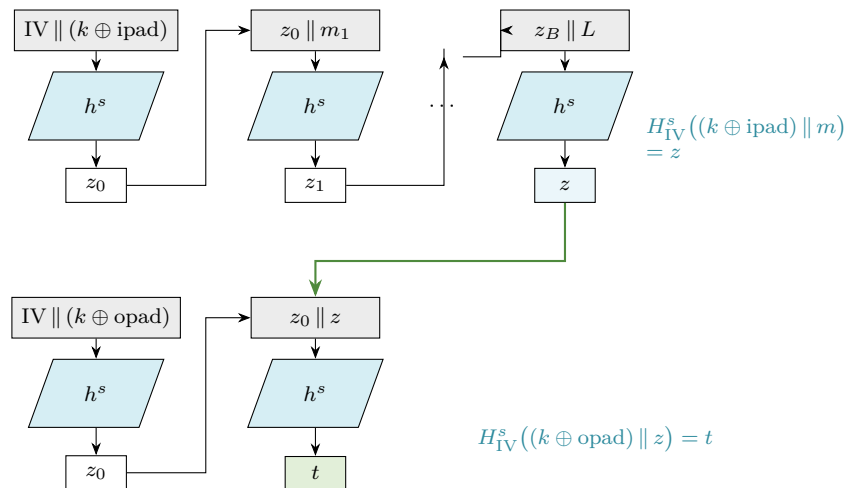


Figura 9: HMAC: hash interno sobre $(k \oplus \text{ipad}) \parallel m$ produce z ; hash externo sobre $(k \oplus \text{opad}) \parallel z$ produce la etiqueta t . (Recreación de la diapositiva práctica.)

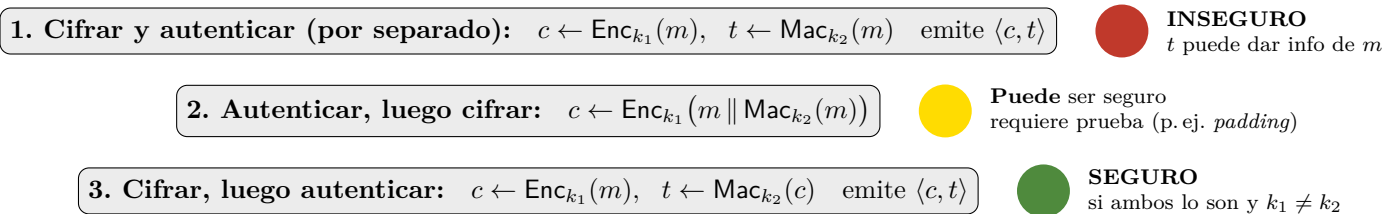


Figura 10: Las tres combinaciones de cifrado y autenticación, con su “semáforo” de seguridad. (Recreación de las diapositivas teórica y práctica.)

- **(3) Cifrar luego autenticar** (Mac sobre el *cifrado*): **siempre seguro**, siempre que los algoritmos de cifrado y MAC lo sean y las claves k_1, k_2 sean **independientes**. (Con claves iguales el esquema puede ser susceptible a ataques.)

11 Cifrado Autenticado

Construcción de cifrado autenticado (encrypt-then-MAC)

Sean $\Pi_e = (\text{Gen}_e, \text{Enc}, \text{Dec})$ un criptosistema **CPA-secure** y $\Pi_m = (\text{Gen}_m, \text{Mac}, \text{Vrfy})$ un MAC **infalsificable** (con etiquetas únicas: para todo k, m hay un único t con $\text{Vrfy}_k(m, t) = 1$). Se define:

Gen: $k_1 \leftarrow \text{Gen}_e$, $k_2 \leftarrow \text{Gen}_m$ (**independientes**)

Enc: $c \leftarrow \text{Enc}_{k_1}(m)$, $t \leftarrow \text{Mac}_{k_2}(c)$ emitir $\langle c, t \rangle$

Dec: si $\text{Vrfy}_{k_2}(c, t) = 1 \rightarrow m = \text{Dec}_{k_1}(c)$; si no $\rightarrow \perp$ (fallo)

El criptosistema resultante es CCA-secure. (Ver Teorema 4.20, Katz & Lindell.) La idea de la prueba es mostrar que el oráculo de descifrado **no le sirve de nada** al adversario (cualquier c adulterado falla la verificación del MAC).

Cifrado autenticado (Secure message transmission):

Privacidad + integridad = CCA-Secure + MAC-forge Secure.

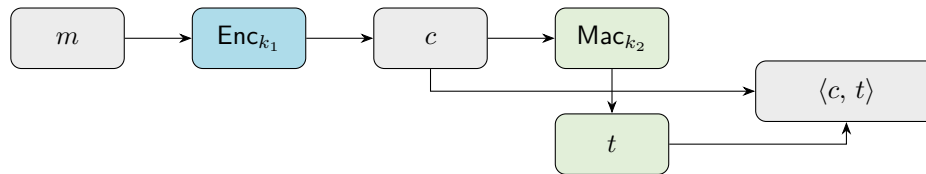


Figura 11: Cifrado autenticado = *encrypt-then-MAC*: privacidad (Enc CPA-secure) + integridad (Mac infalsificable) $\Rightarrow$ **CCA-secure** + **MAC-forge secure**.

11.1 Encadenamiento CCM

CCM — Counter with CBC-MAC

Modo “**Authenticate-then-encrypt**”:

$$e_k(\text{cbc-mac}_k(M) \parallel m).$$

Existe una **prueba de seguridad específica**: si el IV y el *nonce* **no coinciden ni se reutilizan**, la construcción es **CCA-secure usando LA MISMA CLAVE**. (*Ejercicio de la slide: esquematizar un cifrado usando AES-CCM.*)

11.2 Encadenamiento GCM

GCM — Galois/Counter Mode

Forma de encadenar un criptosistema de bloque que **provee cifrado autenticado**. El cifrado es en **modo counter**; la etiqueta de autenticación (MAC) se calcula con **GHASH** sobre $\text{GF}(2^{128})$:

$$H = E_k(0^{128}), \quad \text{Mult}_h(x) = \text{Mult}(x, h), \quad \text{Mult}(x, y) = x \cdot y \bmod (x^{128} + x^7 + x^2 + x + 1).$$

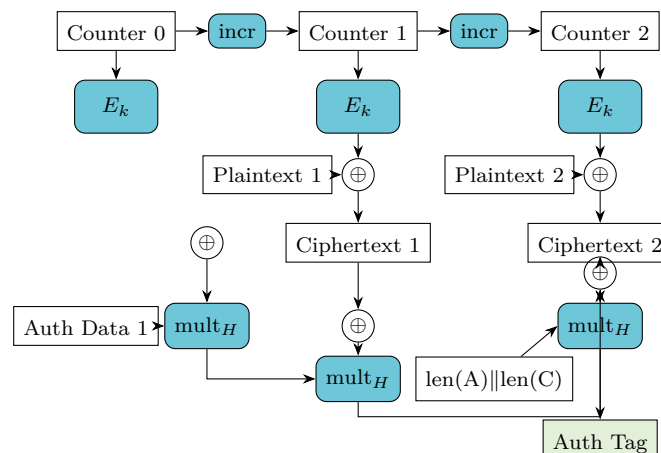


Figura 12: GCM: cifrado en modo counter más cadena GHASH (multiplicaciones en $\text{GF}(2^{128})$) que produce el *Auth Tag*. (*Recreación de la diapositiva.*)

Ejercicio (slide práctica): claves iguales rompen encrypt-then-MAC

Enunciado. Considerar F una función pseudoaleatoria biyectiva fuerte, $\text{Enc}_k(m) = F_k(m \parallel r)$ y $\text{Mac}_k(c) = F_k^{-1}(c)$ (pasa Mac-forge). Probar que el esquema *cifrar y luego*

autenticar no es resistente a CCA por tener claves iguales.

Idea de la resolución. Como $\text{Mac}_k = F_k^{-1}$ y $\text{Enc}_k = F_k(\cdot)$ usan *la misma* clave k , el tag $t = F_k^{-1}(c) = F_k^{-1}(F_k(m \parallel r)) = m \parallel r$ **revela el plano** (más el random). El adversario, al recibir el challenge $\langle c, t \rangle$, lee t y obtiene directamente $m \parallel r$, distinguiendo m_0 de m_1 *sin* siquiera usar el oráculo de descifrado. La moraleja confirma la condición de la slide: **las claves k_1 y k_2 deben ser independientes.**

12 Aplicaciones prácticas

- En términos generales, **usar solo cifrado autenticado** en sistemas reales.
- El cifrado “normal” puede ser **manipulado** (maleabilidad).
- En aplicaciones reales, el requerimiento de **privacidad** lleva implícito el de **integridad**.
- Muy pocas veces se tiene control sobre el material que va a ser cifrado y descifrado: la solución debe funcionar **independientemente** de eso.

Cross-check teoría vs. práctica

Qué agrega la clase práctica

- **Mac-forge explícito sobre los 3 MACs inseguros:** la práctica (y la teórica) resuelven *paso a paso* cómo se rompe $G(k) \oplus m$, $k \oplus \text{first_k_bits}(m)$ y $\text{Enc}_k(|m|)$, destilando las propiedades de un MAC seguro (procesar todo el mensaje, sin determinismos extraíbles).
- **Análisis de las 3 sugerencias para longitud variable** (XOR de bloques, tag por bloque, tag con número de secuencia) con el ataque concreto de cada una (anular el XOR, permutar bloques, mezclar bloques de mensajes distintos en la misma posición). Esto es desarrollo *propio de la práctica*, no de las slides teóricas.
- **Construcción segura $r \parallel L \parallel i \parallel m_i$** y la justificación intuitiva de por qué el random r impide mezclar mensajes (no hay un único r que sirva para un tag híbrido), más la observación de que es **ineficiente**.
- **Ataque a CBC-MAC de longitud variable** explicado vía estados intermedios (t_1, t_2 que se descartan pero se recalculan).
- **Aclaraciones del profesor** ausentes en las slides: el MAC **no provee no-repudio** (clave simétrica) y **no protege contra replay** (usar número de secuencia o timestamp).
- **Ejercicio CCA con claves iguales** ($\text{Enc}_k(m) = F_k(m \parallel r)$, $\text{Mac}_k(c) = F_k^{-1}(c)$) que motiva la independencia de claves.

Lectura recomendada

Capítulo 4

Introduction to Modern Cryptography

Katz & Lindell

Guías de práctica: se sugiere avanzar hasta la **guía 3**.